REHARNESS: Evidence-Driven Reconstruction of Device Drivers from C Source to Multi-Backend Targets

Anonymous Author(s)

Abstract

Translating a Linux device driver to another environment is difficult because its source interleaves operating-system integration with the device core. Rule-based systems such as C2Rust preserve this complete source structure, leaving the result OS-bound and its register protocol implicit; unconstrained LLMs can emit idiomatic target code but may hallucinate register behavior and stale kernel APIs. We present REHARNESS, a hybrid LLM translation framework with a bounded deterministic evidence boundary. A static extractor analyzes the complete Linux driver and records recognized hardware-facing behavior in a structured Register Interaction Sequence (RIS) contract: register identity, width, order, path predicates, typed bus transactions, and selected buffer/state data flow. Backend plugins translate this evidence—never raw C source—into harness C, bare-metal C, Linux kernel modules, and Rust `no_std`; an LLM performs target-language emission, while deterministic compilation, receipt accounting, primitive-shape checks, and exercised-path trace comparison provide bounded acceptance and repair. In our DesignWare APB SSI case study, 1,844 Linux source lines yield 828-line Linux, 581-line bare-metal, and 276-line Rust targets. Evaluation over 19 drivers and 3 multi-source modules separates deterministic baseline compilation from conservative artifact readiness. Two deterministic outputs pass QEMU value/trace oracles, reproduced with the restored rule-based emitters. The DesignWare artifact set is audited by a versioned, machine-checked 18-item checklist: the three C backends pass the control, configuration, and ordered tx/rx data-flow items, while per-backend strict compilation and the Rust buffer data-flow items are reported as open checklist failures rather than claimed as a full pass.

CCS Concepts: • **Computer systems organization** → **Embedded systems**; • **Software and its engineering** → *Software development techniques.*

ASPLOS '27, 2027,

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Keywords: device drivers, evidence intermediate representation, program analysis, code generation, LLM-assisted synthesis, Rust

ACM Reference Format:
Anonymous Author(s). 2026. REHARNESS: Evidence-Driven Reconstruction of Device Drivers from C Source to Multi-Backend Targets. In *Proceedings of ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '27).* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Device drivers constitute a large fraction of operating-system code and account for a disproportionate share of kernel faults [1, 2]. Porting a driver to a userspace test harness, bare-metal firmware, another operating-system environment, or a memory-safe language is therefore a recurring systems problem. Although this task is commonly described as source-to-source conversion, a driver is not a homogeneous program. Its C source interleaves two concerns: *operating-system integration*, including resource acquisition, subsystem registration, callbacks, synchronization, and object lifetimes; and the *device core*, including register interactions, bus transactions, path conditions, and data movement between software state and the device. The latter contains hardware-facing behavior to preserve; the former is re-bound to the target but can still affect it through callbacks and state.

Existing translation approaches handle this mixture poorly. Deterministic translators such as C2Rust [9] preserve the source program structurally and reproducibly, but consequently carry Linux-specific framework code and its pointer-heavy representation into the target. The translated program remains tied to APIs and lifecycles that may not exist outside Linux, while the device protocol remains implicit in syntax. Direct large-language-model (LLM) translation takes the opposite approach: a model can restructure code and emit idiomatic target APIs, but it is also asked to infer which source behavior matters. For a driver, that authority is unsafe. A plausible candidate may add a DMA command, omit an interrupt acknowledgment, alter a register width, or reorder a buffer update while still compiling successfully.

The central question is therefore not whether translation should use rules or an LLM, but *where nondeterminism is allowed.* We present REHARNESS, a constrained hybrid translation pipeline that places the LLM between two deterministic stages. First, static analysis examines the complete Linux

driver—including its framework glue—and records recognized hardware-relevant behavior in a structured evidence contract. The contract is expressed as a Register Interaction Sequence (RIS) together with device and binding specifications. It records recognized register identities, widths, operation order, path predicates, typed bus transactions, and selected software-state/data-flow effects. Second, an LLM translates this compact contract, rather than the original C source, into a backend-specific implementation. Third, deterministic compilation, receipt accounting, and exercised-path trace comparison decide whether the candidate is accepted within the modeled scope.

This separation is semantic, not a source transformation. REHARNESS does not delete Linux glue from the input driver or assume that framework code is irrelevant. The extractor analyzes the full translation units and retains any condition, state binding, call relationship, or resource fact that affects a hardware interaction. What it excludes from the cross-backend contract is framework structure for which no device-core effect is established. Consequently, the LLM is not asked to rediscover the boundary between Linux integration and hardware behavior, nor to reproduce every kernel idiom from memory.

Confining the model to the extracted contract also reduces the translation surface. In the DesignWare APB SSI case study, the pinned Linux source spans 1,844 lines across its core, MMIO glue, and header, whereas the generated targets contain 828 lines for Linux C (source and header), 581 for bare-metal C (source and header), and 276 for Rust `no_std`. This reduction does not prove correctness and is not achieved by modifying the original driver. It arises because each target implements the extracted device core plus only the adapter required by that backend. The smaller structured input and output may leave fewer opportunities to invent unrelated framework APIs or control flow; the current verification gate rejects only deviations observable under the modeled contract and checked traces.

REHARNESS makes five contributions:

- **A device-core extraction pipeline** combining libclang AST analysis, flow-sensitive dataflow and taint tracking, and LLVM-IR evidence at `-O1`. It records recognized hardware-relevant evidence from full Linux driver sources and recovers MMIO operations hidden in header-inlined `static inline` accessors.
- **The RIS evidence language**, extending register operations with typed regmap/I2C/MFD transactions and functional-state operations (`ValueBind`, `StateWrite`, and `OutputWrite`) that record selected buffer-to-register data flow.
- **A constrained hybrid LLM translator** in which four backend plugins—harness C, bare-metal C, Linux kernel module, and Rust `no_std`—share one deterministic evidence contract. The LLM receives the contract and a fixed backend prompt, never the original C source.
- **A bounded verification gate** combining strict compilation, exactly-once accounting for recognized register operations, primitive-shape checks for covered C leaves, and exercised-path trace comparison with bounded structured repair. LLM output is treated as an untrusted candidate rather than an authoritative translation.
- **An artifact-backed evaluation** over 19 single-source drivers and 3 multi-source modules, including deterministic QEMU experiments and one non-repeated four-backend DesignWare APB SSI artifact case with an 18-item checklist. Deferred stronger experiments are listed explicitly with the artifact.

2 Motivation

2.1 Linux Drivers Interleave Two Translation Concerns

The hardware protocol of a Linux driver rarely appears as one isolated block. A `probe` callback acquires MMIO resources and initializes device registers; an interrupt callback combines framework state with status and acknowledgment accesses; a transfer callback advances software buffers around FIFO reads and writes. Register accessors may be macros or `static inline` functions in headers, while the callback that invokes them is registered through a subsystem-specific operations table. Device-core semantics and Linux integration are therefore interleaved across functions and translation units.

A useful cross-backend representation must separate these concerns without pretending that framework code has no effect. For example, the registration of a callback is Linux glue, but the binding between the callback field and the function determines the function's semantic role. Resource acquisition is platform-specific, but it establishes which state field is the MMIO base. A branch in a framework callback may guard a register write and must therefore remain in the extracted contract. REHARNESS analyzes these constructs as evidence while translating only the resulting device-core behavior. This distinction—analyzing the complete source but translating a smaller semantic contract—is the foundation of our hybrid approach.

2.2 Existing Translation Boundaries Are Insufficient

Deterministic source translation. C2Rust-style translation [9] maps C syntax and control flow to a target language using deterministic compiler transformations. This provides reproducibility and broad syntactic coverage, but it preserves the source program's decomposition. A Linux driver's `devm_*` resource calls, callback tables, locking idioms, error-unwind paths, intrusive containers, and kernel object lifetimes all survive in translated form. Outside Linux, many

of those APIs do not exist; inside Rust, their pointer and aliasing structure often remains `unsafe` because the translator lacks the semantic distinction between an MMIO mapping, a software buffer, and an ordinary pointer.

Compiler infrastructures such as LLVM [12] make it practical to analyze and transform large C programs. Code-pretrained models such as CodeT5 [17] improve learned code generation, but whole-program syntactic translation still does not expose the reusable device contract. Macro-defined offsets, wrapper accessors, register-dependent branches, transaction APIs, and buffer/FIFO ordering remain encoded in target-language statements. Retargeting the result to another backend still requires a developer to recover the same hardware protocol manually. Determinism is valuable, but preserving every source-level concern is not the same as preserving the right cross-platform semantics. The prevalence of integration code is measurable: across the 19 driver corpus, only 9.5% of non-blank source lines are referenced by an emitted device-core operation; the remaining 90.5% split into framework entry bodies (32.6%), glue wrappers and other function bodies (25.7%), and file-scope declarations with registration boilerplate (32.2%) (Figure 3).

Direct LLM translation. Code-pretrained models such as CodeT5 [17] can restructure code and select idiomatic target APIs, but direct translation gives the model responsibility for both semantic discovery and code emission. The model must decide which callbacks matter, distinguish MMIO from ordinary memory, resolve macros and wrapper calls, preserve path guards, and reconstruct target-specific framework code at once. The complete Linux source also supplies a large amount of subsystem boilerplate that invites memorized but version-incompatible APIs.

These failures are not merely stylistic. During development, a QEMU edu prompt that prohibited DMA and IRQ setup still produced a candidate that allocated DMA state and wrote `DMA_CMD|DMA_IRQ`; the resulting interrupt storm hung the guest and destroyed the feedback channel. Other candidates used obsolete kernel interfaces or produced different behavior from identical evidence. Compilation cannot detect a type-correct extra register write, a missing acknowledgment, or an incorrectly advanced transfer buffer. Direct LLM translation therefore lacks a stable authority for what the target must preserve.

A deterministic semantic boundary. Hybrid translators such as TransCoder [16] and CoTran [10] combine learned translation with feedback, but verification alone does not define what the model is allowed to reinterpret. If the model still receives the complete driver and owns the extraction of its meaning, the candidate search space includes arbitrary omissions and inventions. Tests can reject observed failures, but they do not by themselves provide a complete translation contract.

Earlier driver-analysis and synthesis systems provide deterministic foundations for such a boundary. SDV checks operating-system API protocols [3]; Devil+ and Dingo/Termite synthesize drivers from explicit device/OS models [4–6, 11]; and KLEE or S2E explore paths when an executable environment exists [7, 8]. These approaches either target framework protocols, require manually authored specifications, or depend on executable models. They do not extract a compact hardware contract from an existing Linux driver as the input to constrained multi-backend translation.

REHARNESS moves the deterministic boundary before generation. AST and LLVM-IR analysis determine the hardware-relevant operations and their provenance; flow-sensitive analysis determines ordering, predicates, and data dependencies; semantic inference supplies device roles and backend bindings. Only this framework-independent evidence is sent to the LLM. The model is instructed to choose how to express an established operation, not whether it exists. After generation, receipts enforce the required traceable register projection, while trace comparison checks expected MMIO order on exercised paths. This follows the broader translation-validation idea [13–15], but applies a narrower, evidence-bound check to generated drivers.

The resulting target is often smaller than the original Linux driver because it contains the extracted device core and a backend-specific adapter rather than a translation of every source construct. This is a reduction in the model's semantic and textual search space, not a deletion of source code and not a correctness argument by itself. Confidence in the modeled properties comes from the deterministic evidence and stated acceptance checks; reduced code size makes the probabilistic step easier to constrain and audit.

3 System Design and Evidence Contract

REHARNESS is structured as a four-stage pipeline that transforms C driver source into verification-gated, backend-specific driver code. Figure 1 shows the flow of artifacts through the system. The key design principle is the *evidence contract*: deterministic analysis establishes the required modeled behavior, and downstream stages are instructed to re-express rather than reinterpret it. The verification gate checks required traceable register operations and exercised traces; it does not claim to exclude every additional hardware access or establish whole-driver equivalence.

3.1 Architecture and Deterministic Boundary

Deterministic AST analysis resolves macros, callback bindings, control flow, and data dependencies; LLVM IR supplements MMIO hidden by header-inlined accessors (Section 4). The result is the RIS evidence IR plus backend-independent function/device summaries. Backend plugins serialize that contract for an LLM, which emits harness C, bare-metal C, Linux C, or Rust `no_std`. Compilation, receipt accounting

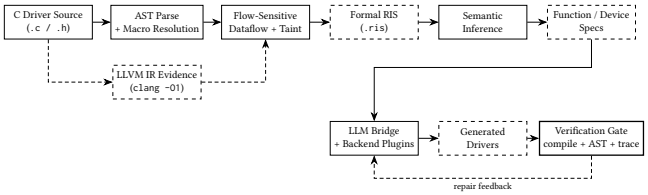

Figure 1. Architecture of REHARNESS. Stage 1 performs hybrid AST+IR extraction into the structured RIS evidence IR. Stage 2 infers backend-independent function and device specifications. Stages 3–4 emit target drivers via pluggable backends driven by an LLM bridge, with a bounded verification gate providing structured repair feedback.

for recognized register operations, and exercised-path trace checks then accept or reject the candidate within their stated scope and provide bounded repair feedback. Across 19 drivers, this pipeline extracts 485 evidenced operations over 157 registers; the multi-source evaluation covers 19 translation units and 27447 lines.

3.2 Register Interaction Sequence

The Register Interaction Sequence (RIS) is the central structured evidence intermediate representation of REHARNESS. It provides a machine-parseable syntax for register, transaction, and functional-state observations, together with typed expressions and nested control-flow evidence. The current artifact does not define a complete denotational semantics or prove whole-program refinement; RIS defines the required evidence only for the operations and paths that the extractor recognizes and records. Every downstream stage consumes this bounded evidence contract.

Grammar and operation tiers. A RIS document consists of a driver header, one or more modules (one per analyzed function), and metadata blocks for accounting and validation. Each module is an ordered sequence of operations. The top-level grammar is:

```
driver <name> v<ver> {
  module <func> {
    <op>            -- one operation per
        source statement
    ...
  }
  ...
  accounting { source_accesses N; emitted N;
      ... }
  ir_analysis { source "f.c" { ir_generated
      true; ... } }
}
```

Operations are organized into three semantic tiers.

Tier 1: Register operations model direct MMIO interactions:

- `R(<width>, <addr>)` — Read: loads a value of the given width from the register address; the result is bound to a variable.
- `W(<width>, <addr>) = <expr>` — Write: stores a computed expression.
- `RMW(<width>, <addr>) = <transform>` — logical Read-Modify-Write: records a read, a transformation expression, and a write-back to the same address. It does not claim hardware-level atomicity.

The width annotation (`B1`, `B2`, `B4`, `B8`) records the access width in bytes, distinguishing a byte-wide status read from a 32-bit configuration write or a 64-bit access. This information is preserved through code generation so that the backend selects the intended primitive (`readb` vs. `readl` in Linux, or the corresponding register field width in Rust `tock-registers`).

Tier 2: Transaction operations model typed bus-level accesses that are not MMIO:

- `TXREAD[<transport>] <target>@<selector>` — reads via regmap, I2C/SMBus, or MFD.
- `TXWRITE` and `TXUPDATE` — writes and masked updates preserving the selected transport's semantics.

The transport field (`regmap`, `i2c_smbus`, `i2c`, or `mfd`) distinguishes the API family, and the target/selector pair keeps the bus handle separate from the register or command selector. This preserves a distinction that a flat MMIO representation would erase, such as the difference between a regmap handle and a memory address.

Tier 3: Functional-state operations capture data flow between software state and device registers:

- `VALUE(<expr>)` — ValueBind: binds a variable from a source-level expression (e.g., a buffer dereference).
- `STATE(<field>) := <expr>` — StateWrite: updates persistent driver state (cursor, counter, flag).
- `OUT(<target>) := <expr>` — OutputWrite: stores a value into a software output buffer.

These three tiers are interleaved in the extractor's source-order projection within each module. The `DELAY(<cycles>)` and `RETURN <expr>` operations round out the set. Each operation carries an `op_id` (sequential identifier), a *reliability* tag (`Exact` or `Conservative`), an *intent* annotation (e.g., `Init`, `Status`, `Config`, `DataTransfer`), and source-location provenance (file and line), providing provenance for recognized statements rather than a complete audit of the original program.

Register address model. Register addresses in RIS are classified into three disjoint categories, forming the address domain $\mathcal{A}$:

$$\mathcal{A} = \underbrace{\textsc{Symbolic}(d, r)}_{\substack{\text{device } d\text{, register } r \\ \text{(macro-resolved)}}} \cup \underbrace{\textsc{Fixed}(b, o)}_{\text{base } b\text{, numeric offset } o} \cup \underbrace{\textsc{Computed}(e)}_{\text{runtime expression } e}$$

A *Symbolic* address is the ideal outcome: the extractor resolved a preprocessor macro to its register name and numeric offset, producing a stable, human-readable identifier such as `dws->regs.DW_SPI_DR`. A *Fixed* address has a numeric offset but no resolved name—for instance, a bare constant written directly at the call site. A *Computed* address carries a runtime expression, such as `base + (1 « (offset + 2))`, used for indexed register banks. The extractor never fabricates a symbolic name for an address it cannot statically resolve; it preserves the expression and marks it as a computed access, which downstream readiness analysis treats conservatively.

Expression algebra. Values use CONST, VAR, binary operators, bit slices, and conditional ITE terms; TOP explicitly represents an unresolved value. Constant folding canonicalizes known subexpressions. RMW extraction represents straight-line and branch-dependent mutations in the same algebra, using nested ITE terms when different guards transform a read value before it is written back.

Control flow. RIS uses `IF/ELSE`, `LOOP`, and `SEQ`. Guards and loop details use the same expression algebra. A predicate stack attaches accumulated `if` and `switch-case` conditions to leaf operations. This preserves path evidence for generation and analysis; the current receipt and runtime checks do not independently prove that a generated guard is equivalent to the source guard on every input.

Example. Listing 1 shows an extracted RIS fragment from the DesignWare APB SSI transmit path, illustrating all three operation tiers in a single loop body.

Listing 1. RIS fragment from `dw_spi_transfer_handler`: the SPI transmit loop, showing functional-state operations interleaved with register writes.

```
LOOP while dws->tx_len {
  IF dws->tx {
    IF (dws->n_bytes == 0x1) {
      txw := VALUE(*(u8 *)(dws->tx))     --
          ValueBind
    }
    STATE(dws->tx) := (dws->tx + dws->
        n_bytes) -- StateWrite: cursor
  }
  W(B4, dws->regs.DW_SPI_DR) = txw
                -- Write: push to FIFO
  STATE(dws->tx_len) := (dws->tx_len + -1)
        -- StateWrite: counter
}
```

The `VALUE` operation records that `txw` is derived from dereferencing the transmit buffer pointer; the two `STATE` operations advance the buffer cursor and decrement the remaining-length counter; and the `W` operation pushes the bound value to the data register. Without the functional-state operations, a generated driver could reproduce the register-write pattern yet fail to transfer data. The contract therefore requires the `VALUE` ⟶ `STATE` ⟶ `W` ⟶ `STATE` order. The general runtime oracle observes only MMIO events; dedicated DesignWare artifact checks inspect this functional-state pattern in the generated targets.

3.3 Device and Function Semantics

RIS specifies recognized ordered hardware-facing evidence; semantic inference adds roles needed to translate that evidence to a target environment. Each `FunctionSpec` records a backend-independent signature, callback role, execution context, state bindings, effects, pre/postconditions, and a reference to its RIS module. Callback-table evidence takes precedence over name heuristics: an `irq_chip.irq_ack` initializer, for example, establishes an interrupt-acknowledgment role even when the function name is ambiguous. C types are abstracted to concepts such as `DeviceState`, `LogicalIRQ`, and `UInt`.

A `DeviceSpec` composes these function contracts with the device class, state, resources, accessed register map, and invariants. MMIO base expressions become device-state bindings; observed callback roles introduce the corresponding IRQ, clock, GPIO, SPI, or bus resources. The result describes the extracted device core without prescribing Linux C, freestanding C, or Rust syntax.

3.4 Backend Bindings and Evidence JSON

A `BindSpec` maps abstract concepts to one backend: types, MMIO primitives, state expressions, callback slots, and exported entry points. Source facts retain supporting information that is useful for reconstruction but is not part of the portable contract, including structure fields, constants, callback initializers, and resource-acquisition evidence. At generation time, REHARNESS serializes the RIS operations, device identity/class, `BindSpec`, and selected source facts into one evidence JSON. `FunctionSpec` roles that cross the boundary are materialized as callback and state bindings rather than serialized as a separate object. This JSON is the only semantic input to the LLM; the original C source is not included.

4 Device-Core Contract Extraction

The extraction stage transforms complete Linux driver sources into the evidence contract defined above. A libclang-based AST path supplies macros, control structure, callback bindings, and flow-sensitive state; LLVM IR supplements operations hidden by header-inlined accessors.

4.1 AST Dataflow and Inter-Procedural Analysis

Translation units and macros. The pipeline constructs a libclang translation unit for each `.c` or `.h` input with

detailed processing records, preserving macro definitions, expansions, and included-header context. A macro table maps register identifiers to integer offsets, allowing `base + GPIO_INT_EN` to become a symbolic register address rather than opaque text.

When a `compile_commands.json` database is available, the parser uses its include paths, definitions, and target arguments; otherwise it records diagnostics and uses the available kernel include configuration.

Target functions include lifecycle entry points, interrupt handlers, and callbacks registered through Linux operations tables. The table type and field are retained so semantic inference can distinguish roles such as interrupt acknowledgment and GPIO direction control.

Flow-sensitive dataflow and taint tracking. For each target function, the extractor performs a flow-sensitive intra-procedural dataflow analysis. It walks the function's statements in source order, maintaining an *abstract store* $\Sigma : \mathit{Var} \to \mathcal{V}$ mapping program variables to abstract values. The abstract value domain $\mathcal{V}$ is designed specifically for register interaction analysis:

- `BasePtr`(b) — a pointer known to originate from `ioremap` or an `__iomem` parameter; the MMIO base.
- `Offset`(b, o, r) — `BasePtr`(b) plus a constant offset o, optionally annotated with a resolved register name r.
- `ReadTaint`(a, r) — a value loaded by an MMIO read from address a; carries a taint label linking it back to the source register.
- `Const`(n) — an integer literal or fully folded constant.
- `SymExpr`(t) — a symbolic expression that cannot be fully evaluated (e.g., a runtime variable or unanalyzable call).
- `Top` — an opaque, unknown value.

The analysis seeds the store with known MMIO sources. When it encounters an `ioremap` or `devm_ioremap` call, the return value is bound to `BasePtr` in the store. Pointer-typed parameters (those with `__iomem` annotation or `void *` type) are also seeded as `BasePtr` candidates. As the walk proceeds, assignments update the store: `g = gpiochip_get_data(gc)` records that `g` derives from `gc`; a subsequent `g->base` access is then resolved through the store to identify the MMIO base field.

Address resolution is the core operation. When the extractor encounters an MMIO call (e.g., `readl(g->base + GPIO_INT_STAT)`), it evaluates the address expression against the current store and macro table. The expression `g->base + GPIO_INT_STAT` is split at the top-level `+`, yielding `g->base` (resolved to `BasePtr` via the store) and `GPIO_INT_STAT` (resolved to `Offset` via the macro table, with `reg_name = "GPIO_INT_STAT"`). The combination produces an `Offset` with the resolved base and numeric offset, plus the symbolic register name. If no macro resolves, the numeric constant (or expression) is preserved as a `Fixed` or `Computed` address.

Branch conditions are tracked through a *predicate stack*. As the walker enters an `if` body, the condition expression is pushed; for the `else` path, the negation is pushed. Each leaf operation inherits the full stack as its guard. Switch-case branches are handled by conjoining the selector-equality predicate for each case. This means the extracted RIS preserves not just *what* register is accessed but *under what condition*—information that is essential for trace-level verification and for generating code that reproduces the same branching behavior.

Read-modify-write detection. RMW patterns are a critical register idiom: a value is read from a register, modified (typically by bitwise OR, AND, or XOR with a mask), and written back to the same address. REHARNESS detects these by matching the taint flow. When a `readl` stores its result into variable v at address a, the extractor records a `ReadTaint` on v linked to a. Subsequent mutations of v (`v |= BIT(n)`, `v &= mask`, etc.) are accumulated. When a `writel` of v (or an expression derived from v) targets the same address a, the read-mutate-write sequence is collapsed into a single `RMW` operation.

The transformation expression is reconstructed by replaying the mutation sequence. For straight-line code, this is a simple composition: each compound assignment extends the expression tree. For branch-dependent mutations, the extractor builds an ITE nest: for each guard g_i under which v is mutated by function f_i, the transform becomes $\textsc{Ite}(g_i, f_i(v_{old}), \ldots)$. Switch-case mutations are handled similarly, generating a disjunction of selector-equality guards. This reconstruction records the RMW transform represented by the extractor without dynamic execution. The current verification gate checks its receipt and primitive-shape obligations only within the stated scope; it does not independently re-prove an arbitrary generated value transform.

Wrapper inlining and inter-procedural analysis. Real drivers wrap MMIO primitives in helper functions. Without inter-procedural analysis, every operation inside a wrapper is invisible. REHARNESS builds a call graph from the parsed AST and performs depth-limited, recursion-safe inlining of callee functions into their callers.

The call graph records, for each function, its callees with argument-to-parameter bindings and control-flow context (the predicate stack at the call site). When a callee is itself a target function with register operations, its extracted RIS operations are instantiated with the actual arguments substituted for formal parameters, and merged into the caller's operation sequence at the call-site position. For recognized inlined calls, this preserves the extractor's source-order projection: if `probe` calls `gpio_setbit` on line 42 and `readl` on line 43, the inlined operations from `gpio_setbit` appear before the line-43 read in the final RIS.

The inlining depth is bounded (default 3) to prevent unbounded expansion of deeply recursive call chains, and a

visited set prevents cycles. The inliner also handles *wrapper summaries*: when a callee is a simple function containing exactly one MMIO call (a read or write) at the top level (no surrounding control flow), REHARNESS infers a compact summary—a mapping from the wrapper's parameters to the underlying MMIO operation—and substitutes it at each call site without a full re-extraction. This is particularly effective for accessor pairs like `dw_readl`/`dw_writel`, where the wrapper is a one-line function forwarding to `readl`/`writel`. The summary inference is conservative: it requires explicit MMIO provenance (an `__iomem` parameter or conventional base-field name) and rejects generic `void *` addresses, reducing misclassification for the wrapper patterns it covers.

For multi-source drivers spanning multiple translation units, the call graph is extended with cross-TU edges resolved through symbol linkage. The multi-source manifest links bitcode via whole-program analysis, resolving 223 cross-translation-unit call edges in the evaluation corpus.

4.2 LLVM-IR Evidence Recovery

The AST path handles macro offsets, wrappers, conditions, and arithmetic addresses, but a further challenge remains in our source-file-oriented libclang path: `static inline` MMIO accessors defined in kernel headers. When a driver's `.c` file calls `dw_readl(dws, DW_SPI_CTRLR0)`, this path records the wrapper call but may not expose the nested `readl` and address computation as a direct MMIO operation in the analyzed function.

REHARNESS addresses this blind spot through a complementary IR pass. The translation unit is compiled to LLVM IR with `clang -O1 -emit-llvm -g`. In the evaluated toolchain, `-O1` inlines the relevant header accessors. Linux MMIO primitives implemented via inline assembly then expand to recognizable patterns:

```
; write: movl $0, $1
tail call void asm sideeffect "movl $0,$1",
    "r,*m,~{...}"(...)
; read:  movl $1, $0
%r = tail call i32 asm sideeffect "movl $1,
    $0", "=r,*m,~{...}"(...)
```

The IR pass scans for `asm sideeffect` instructions matching these templates and classifies each as a read or write. It then extracts three pieces of evidence for each operation.

Offset recovery. The pointer operand is traced backward through preceding `getelementptr` instructions to recover the byte offset from the MMIO base; the `i64` constant in the GEP index yields the numeric register offset. For example, if the GEP is `getelementptr i8, ptr %base, i64 0x60`, the register offset is `0x60`.

Inline chain resolution. Debug-location and subprogram metadata reconstruct the chain from the MMIO primitive through the inlined accessor to the outermost non-primitive caller, attributing each IR operation to its source function.

Source provenance. The source line number is recovered from the outermost `DILocation`, providing provenance matching the AST evidence format.

The recovered IR operations are merged with the AST extraction by a `(function, offset)` key. If the AST path already found an operation at the same offset in the same function, the IR operation is treated as redundant; otherwise it fills an AST blind spot. This conservative merge is useful for the header-inline case, but the key does not encode access multiplicity, source location, call-site instance, or guard. Same-address repeated accesses can therefore collide and are an explicit limitation of the current evidence contract. The merge is additive with respect to distinct keys: no AST operation is removed or rewritten by the IR pass.

IR evidence supplements rather than replaces the AST: IR exposes inlined MMIO, while macro names, predicates, callback bindings, and functional-state operations remain AST evidence. The DesignWare case study combines both paths to extract 244 evidenced operations across 28 modules.

4.3 Functional-State and Typed-Transaction Extraction

Functional state. Beyond register and transaction operations, REHARNESS extracts *functional-state* operations that represent how data moves between software buffers and device registers. They broaden the contract beyond register identity and order, but do not by themselves establish functional equivalence.

The extractor identifies three categories of functional-state operations during the dataflow walk.

ValueBind (`VALUE`). When a local variable is assigned from a source-level expression that feeds a subsequent register write argument, the binding is recorded. For example, `txw = *(u8 *)(dws->tx)` produces `txw := VALUE(*(u8 *)(dws->tx))`. The extractor retains only bindings that are consumed by a later operation (register write, state update, or output store); pure control-flow temporaries remain internal dataflow facts and are not emitted to the RIS module.

StateWrite (`STATE`). When a persistent driver-state field—a struct member accessed via `->` or `.`—is updated, the mutation is recorded. This includes pointer advancements (`dws->tx += n_bytes`), counter decrements (`dws->tx_len-`), flag assignments (`dws->dma_mapped = 0`), and callback registrations (`ctlr->set_cs = dw_spi_set_cs`). Local scalar temporaries that do not escape the function are not emitted. The distinction is important: a state write affects computation visible to subsequent callbacks or inlined helpers, while a local assignment is control-flow bookkeeping. Unary increment

and decrement operations (++/−) on persistent members are also captured, reconstructed as additive assignments.

OutputWrite (OUT). When a value is stored through a pointer dereference into what the analysis identifies as a software output buffer, the store is recorded. For example, `*(u8 *)(dws->rx) = rxw` produces `OUT(*(u8 *)(dws->rx)) := rxw`. The extractor uses the assignment's extent end offset for ordering, ensuring that calls in the right-hand side (such as `*rx = readl(base)`) are emitted before the output write.

These three categories are extracted in source order and interleaved with register operations in the final RIS module. The ordering is critical: the SPI transmit loop's sequence—VALUE (read from tx buffer) → STATE (advance cursor) → W (write to data register) → STATE (decrement counter)—must be preserved, because reordering may write stale data or prevent termination. The evidence JSON retains this order for generation; the current general trace oracle checks the MMIO projection, while case-specific artifact checks cover the associated buffer/state operations.

Typed transactions. Many drivers interact with hardware through bus APIs rather than direct MMIO. Regmap provides a register-access abstraction layer; I2C/SMBus offers byte, word, and block transfers; and MFD (Multi-Function Device) helpers expose per-subdevice register accessors. These calls are semantically distinct from MMIO: a regmap read goes through a bus transaction, not a memory load, and conflating the regmap handle with a memory address would produce unsound specifications.

REHARNESS recognizes public regmap and I2C/SMBus read, write, update, and bulk helpers, preserving the handle as target and register/command as selector. MFD recognition is provenance-gated to declarations under `include/linux/mfd/` and narrow register-helper patterns, preventing arbitrary calls from being misclassified as device transactions.

The transaction operations preserve the bus-level semantics that flat MMIO representations lose: a `TXUPDATE[regmap]` with a mask and value is not equivalent to a read-modify-write on a memory address, because the regmap subsystem may apply caching, buffering, or bus-specific framing. By keeping these as distinct typed operations, RIS ensures that generated backends use the correct API family for each access.

5 Constrained LLM Translation and Verification

The extraction and semantic-inference stages produce RIS, function and device semantics, backend bindings, and selected source facts. This section defines the subset that crosses the LLM boundary, then describes backend plugins, acceptance checks, and bounded repair.

5.1 Evidence Boundary

All backends use one bridge that serializes six JSON sections: *driver* (identity and class), *registers* (name, offset, width), *modules* (ordered operations and nested control flow), *bind* (types, primitives, state expressions, callbacks, and includes), and optional *constants* and *structs* from selected source facts. Module entries retain `op_id`, address, value/transform, transaction transport, and functional-state operations. Function roles are represented by callback and state bindings; the full `FunctionSpec` object is not serialized.

The LLM never sees the original C source. The JSON is a generation-oriented projection, not a claim that every analysis fact crosses the boundary. The prompt authorizes target-language expression of required evidence but forbids inventing or omitting device operations. Subsequent checks detect missing or mutated traceable register operations and compare exercised MMIO order; they do not prove that arbitrary extra accesses are absent.

5.2 Backend Plugins and Prompted Translation

Each backend is a self-contained directory containing a Python module that constructs its `BindSpec` and a `prompt.md` with target rules. A registry discovers modules exposing `NAME` and `generate()`, so adding a target does not modify extraction or verification. **Harness C** uses fake MMIO and access logs; **bare-metal C** uses volatile primitives; **Linux C** emits platform or PCI lifecycles and kernel MMIO; and **Rust `no_std`** uses `tock-registers`, confining `unsafe` to base-address construction.

The bridge instantiates the selected prompt with the evidence JSON, calls the configured endpoint, and extracts candidate code. For example, the Linux prompt selects PCI or platform resource APIs and requests MMIO tracing, while the Rust prompt requires the corresponding `tock-registers` layout and bitfield macros. These rules constrain target adaptation but do not replace the evidence as the source of device operations.

The current implementation invokes a configurable endpoint through `tools/pi/pi_synth.sh`; the bridge interface admits any endpoint that accepts a prompt and returns text. Swapping models does not change the stated acceptance criteria: different candidates face the same compile, receipt, and trace checks.

5.3 Compilation, Receipts, and Trace Oracles

LLM-generated code cannot be trusted. Models may hallucinate APIs, omit or reorder required operations, or introduce accesses absent from the contract. REHARNESS therefore treats every candidate as untrusted and checks it against RIS. Passing means satisfying the checks described below; it does not mean that every unmodeled effect has been excluded. Figure 2 summarizes this boundary and its explicit non-claims.

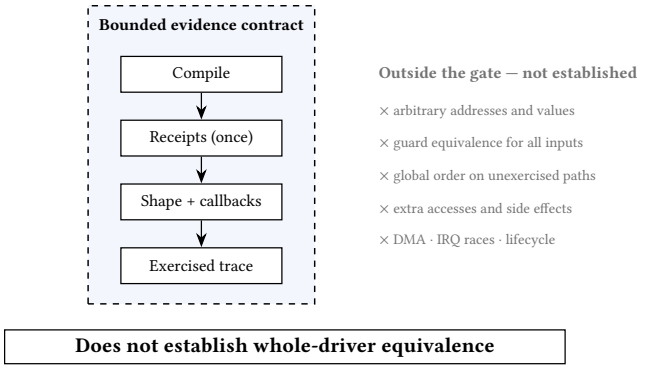

Figure 2. Scope of the verification gate. The left column lists checks implemented by the artifact; the right column records properties intentionally outside the gate. These are bounded evidence checks, not complete verification.

Compilation. The first stage compiles the candidate with strict warnings for the target backend. Harness output is compiled with `cc -Wall -Werror`; bare-metal output with `cc -ffreestanding -Wall -Werror`; Linux output through the pinned kernel's Kbuild interface (`make M=...`); and Rust output with `rustc`. Compilation catches type mismatches, undefined symbols, missing includes, and syntactic errors—the most common LLM failure modes. A candidate that fails to compile is rejected before more expensive checks run.

AST lowering receipts. The second stage is a structural accounting check. Each recognized operation receives a unique `op_id`; generated C carries a lowering marker with that identifier, a canonical contract digest, and an emitted kind. The lowering oracle checks marker presence, cardinality, unknown identifiers, status, kind, and digest consistency with the frozen RIS contract. A separate libclang pass checks C labels and the primitive shape (kind, width, byte order, and write semantics) for covered leaf operations. The current AST oracle does not independently recover arbitrary address or value expressions, guard semantics, or cross-operation ordering from generated code; those are outside the proof scope and must not be described as digest re-computation from the candidate AST.

The receipt reconciliation enforces four properties:

- **Presence:** every RIS operation with a traceable address must have a corresponding receipt. Missing operations indicate that the LLM dropped evidence.
- **Exactly-once:** each `op_id` may appear in at most one receipt. Duplicate receipts indicate redundant or duplicated operations.
- **Kind matching:** a RIS `Read` must lower to a read primitive call, a `Write` to a write primitive call, and a `ReadModifyWrite` to either a read followed by a write at the same address or a single write whose value derives from a prior read of that address (validated through the lowering recipe).
- **Digest consistency:** the digest in the generated marker must match the canonical digest in the frozen contract. This catches marker-level drift, but is not by itself an independent proof that the candidate's address, value expression, guard, or control-flow structure equals the source.

Operations whose addresses are untraceable (e.g., `Computed` addresses with non-lowerable terms) are excluded from receipt reconciliation and instead recorded as explicit readiness blockers, ensuring that the check is sound: it does not credit an operation that cannot be statically resolved to a register offset.

Trace oracle. The third stage compares the generated code's runtime register-access trace against an oracle derived from the structured RIS evidence. The trace oracle operates at two levels. For the harness backend, the generated program is executed and its `[trace]` log is parsed into an ordered sequence of (operation kind, offset) pairs. The oracle derives the expected sequence from the RIS modules' operations, resolving symbolic register names to offsets via the register map and expanding RMW operations into read-then-write pairs.

For Linux and bare-metal backends deployed in QEMU, the trace oracle consumes an instrumented kernel serial log. The instrumentation macros (injected by the Linux prompt template) emit `[rhfn]` *function* boundaries before each module function and `[rh] R/W 0x`*O* `0x`*V* for each MMIO access. The oracle parses these into per-function segments and matches each segment against the expected RIS operation sequence using subsequence matching, which allows the runtime trace to contain additional accesses (e.g., framework-level reads) without failing the check, as long as all expected operations appear in order.

The trace oracle also handles branch sensitivity. The RIS records conditional operations with then/else branches, but the runtime trace reflects only the path actually taken. The oracle computes branch-sensitive sequence variants and accepts a trace matching one of them. Because the current runtime log does not carry a proof of the source predicate's value, this check confirms an exercised sequence variant rather than guard equivalence for all inputs.

Scope of acceptance. The QEMU matcher uses subsequences because framework code may perform accesses outside the extracted callback contract. It therefore confirms that expected MMIO events occur in order on an exercised path, but does not reject every additional access or prove that an unmodeled DMA/IRQ/MMIO side effect is absent. The general runtime trace also does not observe `VALUE`, `STATE`, or `OUT`; those operations are retained in evidence and checked only by dedicated artifact tests in the DesignWare case study.

These limitations are part of the reported readiness boundary.

5.4 Bounded Repair

If any verification stage fails, REHARNESS formats structured feedback and re-prompts the LLM. The feedback identifies the specific failure: a missing receipt for `op_id` n, a kind mismatch (expected `Write`, got `Read`), a compilation error at line L, or a trace mismatch (missing operation at offset 0xO). The LLM receives the current candidate and the failure report, and is instructed to produce a patched candidate. This repair loop iterates until the candidate passes all checks or the iteration budget (default 3) is exhausted, at which point the failure is recorded as a readiness blocker.

6 Evaluation

6.1 Experimental Setup and Research Questions

We evaluate REHARNESS on 19 versioned Linux driver sources spanning GPIO controllers, clock/PLL, AHCI SATA, SDHCI MMC, virtio-MMIO, a framebuffer raster engine, and the QEMU edu PCI device. All results are produced by the machine-readable `./run.sh` pipeline and exported to `generated_results.tex`; the tables in this section are populated by the macros and table macros defined therein, not manually transcribed.

The evaluation addresses five research questions:

- **RQ1 (Address resolution):** Among emitted addressable operations, what fractions are symbolic, fixed, or computed?
- **RQ2 (Extracted structure):** How many RMW patterns and branch conditions does the extractor report?
- **RQ3 (Baseline translation):** Do the deterministically generated harness, bare-metal, and Linux outputs compile?
- **RQ4 (Strict readiness):** Beyond compilation, do the generated outputs satisfy the stated verification criteria (AST receipts, lowering reconciliation, and registration attestation)?
- **RQ5 (End-to-end validation):** Do generated drivers satisfy runtime oracles and, for the LLM case study, the reported case-specific artifact checks?

6.2 Contract Extraction Characteristics (RQ1–RQ2)

Table 1 shows per-driver extraction metrics for a representative subset. Across all 19 drivers, REHARNESS extracts 485 operations: 366 symbolic (77.7% of all addressable operations), 64 fixed, and 41 computed-address accesses. The pipeline detects 73 read-modify-write patterns and 123 branch conditions, referencing 157 distinct registers.

The symbolic rate of 77.7% shows that preprocessing records and flow-sensitive dataflow recover names for most emitted addressable operations. The remaining emitted accesses

Table 1. Extraction results for representative drivers. Sym = symbolic addresses, Fixed = fixed offset, Comp = computed address, RMW = read-modify- write, Cond = branch conditions, Regs = distinct registers. The subset spans the QEMU-validated drivers (ftgpio010, edu), the most condition-heavy driver (virtio_mmio), and the extremes of the symbolic-rate range; per-driver rows for all 19 drivers are in the versioned matrix artifact. In the total row, Ops includes 0-unknown plus 14 non-address operations, so Sym+Fixed+Comp (471) is the addressable denominator of %Sym; per-driver rows have no non-address operations.

Driver	Ops	Sym	Fixed	Comp	RMW	Cond	Regs	%Sym
gpio-ftgpio010	35	35	0	0	7	8	13	100.0%
virtio_mmio	59	46	12	1	0	44	31	78.0%
gpio-pl061	31	27	0	4	7	1	7	87.1%
gpio-cadence	32	32	0	0	4	8	12	100.0%
edu	5	3	2	0	0	2	3	60.0%
Total (19)	**485**	**366**	**64**	**41**	**73**	**123**	**157**	**77.7%**

are classified as 64 direct numeric offsets or 41 runtime expressions for indexed register banks. The aggregate has 471 addressable and 14 non-address emitted operations; the zero unknown count (0) is an internal provenance count, not evidence that no operations were missed. These are internal characterization metrics, not precision or recall against a manually labeled ground truth; an access missed before emission would not appear in the unknown count.

The gpio-ftgpio010 and gpio-cadence drivers achieve 100% symbolic rates because all their register offsets are defined through preprocessor macros that the MacroTable resolves. The edu driver has the lowest symbolic rate (60%) because two of its five accesses use fixed numeric offsets in the original source. virtio_mmio exercises the most complex extraction: 59 operations across 44 conditions, reflecting the virtio configuration-space negotiation protocol.

The same corpus also quantifies the two interleaved concerns of Section 2: classifying each driver's non-blank lines by provenance, the aggregate composition is 9.5% device-core lines, 32.6% framework entry bodies (non-core lines inside callback and lifecycle functions), 25.7% glue wrappers and other function bodies, and 32.2% file-scope declarations and registration boilerplate (Figure 3). Only the first category crosses the cross-backend contract; the remaining three are re-bound per backend. The corpus is integration-dominated, which is precisely the composition the semantic-boundary design targets.

6.3 Translation and Strict Readiness (RQ3–RQ4)

Table 2 reports readiness metrics for the representative subset (selection criteria as in Table 1). Under the frozen deterministic-emitter baseline, the harness backend compiles for 19/19 drivers, the bare-metal backend for 19/19, and the Linux backend for 18/19. All three backends compile the gpio-ftgpio010, gpio-pl061, gpio-cadence, virtio_mmio, and edu drivers. The

Figure 3. Composition of non-blank lines per driver, measured by source-line provenance of emitted operations. Device-core lines are referenced by ≥1 recorded operation; framework entry bodies are the non-core lines of callback and lifecycle functions; glue wrappers and helpers are remaining function bodies; file-scope lines are declarations, macros, and registration tables outside functions (blank lines excluded). The aggregate is 9.5% device-core versus 90.5% integration. Line-level attribution is an approximation: a line interleaving a guard and a register access counts as device-core.

Table 2. Readiness results. RIS = internal RIS quality heuristic (0–1), FuncSpec = function-spec quality, DevSpec = device-spec quality. H/B/L = harness / bare-metal / Linux compilation. Linux ready denotes strict artifact readiness; synthesis-eligible denotes sufficient context to attempt LLM synthesis.

Driver	RIS	FuncSpec	DevSpec	H/B/L compile	Linux ready	Synth.-eligible
gpio-ftgpio010	1.000	1.000	1.000	✓/✓/✓	✓	✓
virtio_mmio	0.897	0.875	1.000	✓/✓/✓	–	–
gpio-pl061	0.924	1.000	0.938	✓/✓/✓	–	✓
gpio-cadence	1.000	1.000	1.000	✓/✓/✓	–	✓
edu	0.920	1.000	0.688	✓/✓/✓	–	✓

single Linux compilation failure is the sdhci-esdhc-mcf driver, whose SDHCI accessor contract lacks a passing source-contract oracle.

The mean RIS quality score across all drivers is 0.902. This is an internal completeness/readiness heuristic, not a calibrated probability or an accuracy estimate against source-level ground truth. The drivers that achieve 1.000 (gpio-ftgpio010, gpio-cadence) have all-symbolic addresses, no unsupported control-flow transfers, and callback bindings that pass the available internal attestation checks.

Strict artifact readiness. Compilation is necessary but not sufficient. REHARNESS distinguishes *compilation* from *strict artifact readiness*: the latter requires that, for the recognized contract scope, every traceable register operation carries a lowering receipt with exactly-once enforcement, that the AST-receipt reconciliation succeeds with no missing or duplicate operations, that callback entries have verified table bindings, and that no readiness blockers (unsupported control-flow transfers, unsafe dynamic addresses, unproven loop summaries) remain.

Under strict artifact readiness, the harness and bare-metal backends achieve 4/19 and 4/19 respectively, and the Linux backend achieves 2/19. The passing drivers are named in the versioned matrix artifact: edu, gpio-ftgpio010, gpio-idt3243x, gpio-pl061 for harness and bare-metal, and gpio-ftgpio010, gpio-idt3243x for Linux. The gap between compilation (18–19/19) and strict readiness (2–4/19) is intentional: REHARNESS treats unknown transforms, unsafe dynamic addresses, and unbound subsystem state as blockers rather than silently

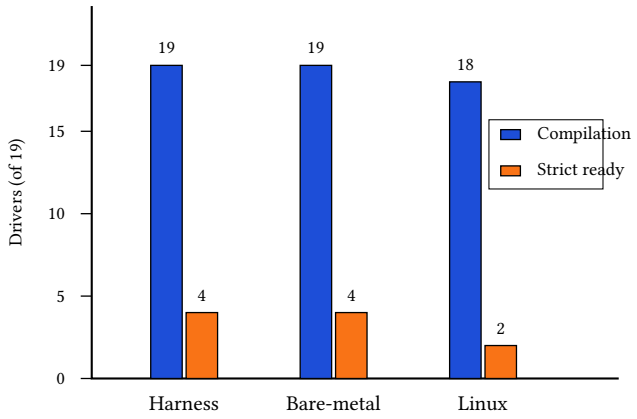

Figure 4. Compilation versus strict artifact readiness across the 19-driver evaluation. Blue bars count compilable deterministic outputs; orange bars count artifacts passing the modeled strict gate. Compilation is necessary but not sufficient, and neither result establishes whole-driver equivalence.

Table 3. Multi-source results. TUs = translation units, LoC = source lines, Funcs = functions analyzed, X-TU = resolved cross-TU call edges, Prop = propagated MMIO edges, Src MMIO = source-level MMIO primitives, RIS MMIO = propagated RIS MMIO operations, H/B/L = backend compilation.

Driver module	TUs	LoC	Funcs	X-TU	Prop	Src MMIO	RIS MMIO	Ops	H/B/L	Orig. Kbuild
aspeed-vhub	5	3540	92	53	54	101	154	158	✓/✓/✓	✓†
c67x00	4	2239	89	30	79	2	32	38	✓/✓/✓	✓†
dwc2	10	21668	445	140	445	804	3608	4250	✓/✓/✓	✓†

approximating them. A driver that compiles but has an unproven loop summary or a registration attestation failure is reported as not ready rather than ready with caveats, ensuring that downstream consumers do not mistake compilation for semantic correctness. Figure 4 makes this compilation/readiness separation explicit.

The 5/19 synthesis-eligible metric counts drivers whose structured context is sufficient for attempting assisted LLM synthesis: their evidence JSON passes the eligibility checks for bindings, unsupported operations, and source facts needed to attempt a candidate. This label does not imply that a candidate was generated or accepted.

6.4 Multi-Source Scalability

To evaluate whether the pipeline scales beyond single-file drivers, we apply REHARNESS to 3 real multi-source Linux USB modules totaling 19 translation units and 27447 lines of code (Table 3). The multi-source pipeline links per-TU bitcode, resolves cross-TU call edges through a single whole-program-analysis (WPA) pass, and propagates MMIO operations through inter-procedural call graphs to bounded depth.

In the three modules, the pipeline analyzes 626 functions and resolves 223 of the 223 cross-TU edges identified by its

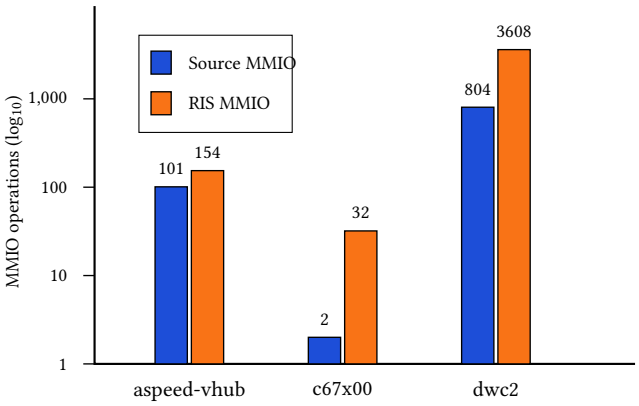

Figure 5. Source MMIO primitives and propagated RIS MMIO operations for the three multi-source modules (log scale). Counts show recorded evidence after propagation; they are not precision/recall measurements and do not establish a complete hardware-access footprint.

linker analysis; module-internal calls are partially resolved: 68 internal call sites remain unresolved (all in dwc2), for an overall call-edge resolution rate of 93.0%. MMIO propagation expands 907 source-level primitives into 3794 recorded RIS operations. This is evidence of wrapper propagation, not a recall measurement or proof of the full hardware footprint. Figure 5 shows the per-module distribution on a logarithmic scale. The dwc2 controller, with 445 functions and 726 call edges across 10 translation units, produces 4250 operations (the three modules total 4446) and is the largest driver in the evaluation; its extraction takes 176.452 seconds.

All three multi-source generated artifacts compile for all three backends (✓/✓/✓), and each module original Kbuild Makefile builds successfully against the pinned kernel tree (indicated by ✓† in the table). The latter is a build-integrity check on the original module and is not a semantic validation of the generated driver.

6.5 Runtime and End-to-End LLM Validation (RQ5)

Two QEMU experiments check concrete runtime behavior of deterministically generated drivers. Neither experiment invokes an LLM; both use the deterministic-emitter baseline (restored as an explicit in-tree mode, `REHARNESS_GENERATION=rules`, so the reported numbers are regenerable). The QEMU suite currently attempts exactly these two experiments; no other corpus driver has a validated runtime manifest, so 2/2 reflects the attempted denominator rather than a corpus-wide pass rate.

QEMU edu (PCI). The deterministic PCI backend generates a miscdevice providing positional reads and writes over the mapped BAR. The guest exerciser validates the device ID at offset 0x00, the complemented live-check value at 0x04, and the factorial result at 0x08. All value-level checks pass (`EDU_TRACE_OK`).

QEMU gpio-ftgpio010 (platform). The experiment registers a platform device, loads the instrumented deterministic module, and runs a GPIO chardev exerciser that triggers generic-chip direction, get-multiple, and set-multiple callbacks. Instrumentation emits exact runtime-function boundaries (`[rhfn]` events), and the oracle matches each selected call segment once against the structured RIS evidence operations, preventing one global trace from being credited to multiple callbacks. The oracle passes (`TRACE_MATCH_OK`) with 6/6 module coverage, 7/7 call coverage, 13/13 operation coverage, and 8/8 register coverage.

These experiments validate two complementary properties: concrete read/write *values* on a modeled PCI device, and access-order and offset *preservation* across a Linux platform-driver probe and exercised subsystem callbacks. Together they show agreement with two exercised protocol slices: concrete values on edu and per-callback access order on gpio-ftgpio010.

End-to-end LLM translation: DesignWare APB SSI. The Synopsys DesignWare APB SSI SPI controller exercises every component of REHARNESS simultaneously: the hybrid IR-enhanced extraction (its register accessors are `static inline` header functions invisible to the AST path alone), the functional-state operations (its transmit/receive paths move data between memory buffers and the data FIFO register), and the prompt-driven four-backend generation with LLM emission subject to the stated verification checks.

IR-enhanced extraction. Header-defined accessors and interrupt-mask helpers hide the underlying MMIO from a source-file-only AST pass. Merging IR evidence by function and offset yields 28 modules and 244 evidenced operations covering control, interrupt, and FIFO registers.

Functional-state preservation. The writer preserves the `VALUE`→cursor `STATE`→FIFO `W`→length `STATE` order shown in Listing 1; the reader preserves the symmetric register-to-buffer `OUT` path. Width-dependent buffer accesses remain distinct branches.

Four-backend generation. All four backends—harness C, bare-metal C, Linux kernel module, and Rust `no_std` with `tock-registers`—were generated by the LLM bridge from this single RIS specification. checked artifacts are audited by a versioned, machine-checked 18-item artifact checklist (`qa/verification/dw_apb_ssi_checklist.py`) covering per-backend compilation, chip-select assertion in `set_cs`, interrupt mask/unmask sequences, transfer configuration, and the ordered tx/rx buffer data flow described above. The checklist result is reported honestly: the three C backends pass all control, configuration, register-order, and C data-flow items; the current versioned Linux artifact fails strict Kbuild compilation against the pinned kernel (an emitted struct member is missing), and the Rust artifact fails the strict-crate build and the six buffer data-flow items because its emission binds buffer reads to constants rather than dereferencing

software buffers. This is an artifact-level case study rather than a repeated-sampling study or a held-out evaluation. The checklist is case-specific and does not establish whole-driver semantic equivalence. The artifacts, checklist script, and per-item results (including open failures) are versioned under `examples/dw-apb-ssi/` and `research/experiments/results/dw-apb-ssi-checklist.json`.

Two observations from this case study shaped the framework design. First, IR evidence is a supplement, not a replacement: in this case the IR pass recovers header-inlined MMIO, but macro names, controlling predicates, and callback bindings exist only at the AST level. Second, register-level equivalence is insufficient for a data-moving driver. An earlier iteration that dropped the `STATE`/`VALUE`/`OUT` operations produced backends that touched the correct registers in the correct order yet could not transfer a byte, because buffer cursors never advanced. This experience directly motivated the functional-state layer in RIS.

7 Discussion and Limitations

7.1 Analysis and Subsystem Boundaries

REHARNESS models common GPIO, IRQ, platform, PCI, AMBA, virtio, clock, power-management, SPI, USB, file-operation, regmap, I2C/SMBus, and MFD paths. Network, DMA-engine, display, and regulator lifecycles still require models. Recursion, backward `goto`s, unbounded loops, and general abstract-store fixpoints remain readiness blockers; optional SVF is outside the default configuration.

Computed register-bank addresses are retained as runtime expressions and lowered when every term is bound. Expressions containing unknown calls, unbound members, or `Top` remain blockers rather than being assigned fabricated symbolic names. Across the evaluation, 41 of 485 operations use this computed-address representation.

7.2 What the Verification Gate Establishes

Compilation is separate from strict artifact readiness. Under the deterministic baseline, harness and bare-metal outputs compile for 19/19 drivers and Linux for 18/19; only 2/19 Linux outputs satisfy modeled readiness. Unknown transforms, unsafe addresses, unbound state, and incomplete lifecycles are blockers, not source-level equivalence claims.

LLM output may violate prompts or vary across models and samples. Compilation, receipt accounting, primitive-shape checks, and exercised traces constrain the reported scope only; they do not reject every extra side effect or prove guard equivalence and arbitrary hardware equivalence. The trusted base is the pinned toolchain, extractor, frozen JSON, receipt producer, and oracle; malicious devices, DMA corruption, interrupt races, and lifecycle failures are out of scope.

Functional-state operations cover selected cursors, counters, and output stores, but the general MMIO oracle does not observe them; only the DesignWare artifact checks inspect this data flow. Interrupt-spanning state machines, DMA lifecycles, concurrency, and physical SPI validation remain open.

7.3 Scalability and Threats to Validity

Parsing kernel translation units dominates runtime (measured per-driver extraction takes 6.0–11.8 seconds, median 9.6, on the evaluation machine; the three multi-source pipelines take 28.5–176.452 seconds each). The multi-source evaluation covers 19 units and 27447 lines; whole-kernel analysis would need persistent caches and parallel execution.

Internal validity depends on libclang, macro resolution, and IR merging; the `(function, offset)` key can collide for repeated same-address accesses. QEMU covers two deterministic protocol slices. The corpus lacks network drivers and physical-hardware validation, and the single LLM case lacks model and repeated-trial metadata. RIS quality, synthesis eligibility, and readiness are internal metrics, not precision/recall. Required follow-up evidence is listed in `EXPERIMENT_DEBT.md`.

A frozen zero-shot holdout (12 drivers excluded from all implementation work, enforced by an in-tree generalization guard) is versioned with the artifact and currently reports 12/12 drivers compiling on all three deterministic backends, strict readiness of 7/12 (harness and bare-metal) and 5/12 (Linux), with 5 cases strict on all backends and 11 cases containing modeled hardware interactions; these results are not yet integrated into the main evaluation and remain an artifact-side generalization datapoint.

8 Conclusion

We presented REHARNESS, a constrained hybrid translator that extracts recognized device-core evidence from complete Linux drivers. AST–IR analysis recovers selected register, transaction, and state operations; an LLM emits four backend forms, which are accepted only within the stated compile, receipt, shape, and exercised-trace checks.

Across 19 drivers and 3 multi-source modules, the results support an evidence-boundary prototype, not whole-driver equivalence; stronger claims remain contingent on the deferred experiments recorded with the artifact.

References

[1] A. Chou, J. Yang, B. Chelf, S. Hallem, and D. Engler. An empirical study of operating system errors. In *Proc. 18th ACM Symp. on Operating Systems Principles (SOSP)*, 2001.

[2] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy. Recovering device drivers. *ACM Transactions on Computer Systems*, 24(4), 2006. doi:10.1145/1189256.1189257.

[3] T. Ball, E. Bounimova, B. Cook, V. Levin, J. Lichtenberg, C. McGarvey, B. Ondrusek, S. K. Rajamani, and A. Ustuner. Thorough static analysis of device drivers. In *Proc. 1st ACM SIGOPS/EuroSys European Conf. on Computer Systems (EuroSys)*, 2006.

[4] L. Ryzhyk, P. Chubb, I. Kuz, and G. Heiser. Dingo: Taming device drivers. In *Proc. 4th ACM European Conf. on Computer Systems (EuroSys)*, 2009.

[5] L. Ryzhyk, P. Chubb, I. Kuz, E. Le Sueur, and G. Heiser. Automatic device driver synthesis with Termite. In *Proc. 22nd ACM Symp. on Operating Systems Principles (SOSP)*, 2009.

[6] L. Ryzhyk, A. Walker, J. Keys, A. Legg, A. Raghunath, M. Stumm, and M. Vij. User-guided device driver synthesis with Termite. In *Proc. 11th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2014.

[7] C. Cadar, D. Dunbar, and D. R. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proc. 8th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2008.

[8] V. Chipounov, V. Kuznetsov, and G. Candea. S2E: A platform for in-vivo multi-path analysis of software systems. In *Proc. 16th Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011.

[9] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf. Translating C to safer Rust. In *Proc. ACM on Programming Languages*, 5(OOPSLA), 2021. doi:10.1145/3485498.

[10] P. Jana, P. Jha, H. Ju, G. Kishore, A. Mahajan, and V. Ganesh. CoTran: An LLM-based code translator using reinforcement learning with feedback from compiler and symbolic execution. In *Proc. 27th European Conference on Artificial Intelligence (ECAI)*, 2024. doi:10.3233/FAIA240968.

[11] Y. Yu, M. Zhu, and S. Chen. New approach for device driver development – Devil+ language. In *Proc. 5th Int'l Conf. on Embedded Software and Systems*, 2005, pp. 418–422. doi:10.1007/11535409_60.

[12] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proc. Int'l Symp. on Code Generation and Optimization (CGO)*, 2004, pp. 75–86. doi:10.1109/CGO.2004.1281665.

[13] G. Barthe, B. Gregoire, C. Kunz, and T. Rezk. Certificate translation for optimizing compilers. *ACM Transactions on Programming Languages and Systems*, 31(2), 2009, pp. 1–45. doi:10.1145/1538917.1538919.

[14] J. Kang, Y. Kim, Y. Song, J. Lee, S. Park, M. D. Shin, Y. Kim, S. Cho, J. Choi, C.-K. Hur, and K. Yi. Crellvm: Verified credible compilation for LLVM. *ACM SIGPLAN Notices*, 53(4), 2018, pp. 631–645. doi:10.1145/3296979.3192377.

[15] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr. Alive2: Bounded translation validation for LLVM. In *Proc. 42nd ACM SIGPLAN Int'l Conf. on Programming Language Design and Implementation (PLDI)*, 2021, pp. 65–79. doi:10.1145/3453483.3454030.

[16] M.-A. Lachaux, B. Roziere, L. Chanussot, and G. Lample. Unsupervised translation of programming languages. arXiv:2006.03511, 2020.

[17] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *EMNLP*, 2021, pp. 8696–8708. doi:10.18653/v1/2021.emnlp-main.685.